DreamsPOS ERP

Documentation Technique — Frontend Angular

Angular 21 TypeScript 5 Standalone Components JWT + 2FA Lazy Loading Bootstrap 5 Tabler Icons

Version : 1.0.0 | **Date :** Juin 2026 | **Auteur :** Équipe DreamsPOS | **Statut :** En cours — Modules 5 & 6 terminés

Table des matières

1. Architecture générale
2. Structure des fichiers
3. Conventions et patterns
4. Intercepteurs HTTP
5. Module 5 — Authentification
6. Module 6 — Config Système
7. Routing et guards
8. Sidebar et navigation
9. Modules à venir

1. Architecture générale

PRINCIPES DIRECTEURS

- **Standalone components** — aucun NgModule, imports déclarés sur chaque composant
- **Isolation layer** — tout le code custom dans `src/_api/`, les fichiers du template touchés au minimum
- **Lazy loading** — chaque groupe de routes chargé à la demande via `loadComponent()`

- **Functional interceptors** — pas de classes, fonctions pures injectées dans `app.config.ts`
- **Services providedIn: 'root'** — singletons, pas de providers dans les composants

```
Navigateur → Angular Router → authGuard → Composant (lazy) → Service (_api/services/) →
HttpClient
                                                    ↑ ↓
                                        jwtInterceptor → errorInterceptor →
apiUnwrapInterceptor
```

Stack technique

ÉLÉMENT	TECHNOLOGIE	VERSION
Framework	Angular	21.x
Langage	TypeScript	5.x
CSS Framework	Bootstrap	5.x
Icônes	Tabler Icons (`ti ti-*`)	—
Template de base	DreamsPOS Admin Template	—
HTTP	Angular HttpClient + intercepteurs fonctionnels	—
Auth	JWT (access + refresh) + TOTP 2FA	—
Build	Angular CLI / ng serve	—

2. Structure des fichiers

```
src/ ├── _api/ # Isolation layer — tout notre code ici │ ├── config/ │ │ └── api.config.ts
# URLs de tous les endpoints (AUTH, SYSTEM…) │ ├── interceptors/ │ │ ├── api-
unwrap.interceptor.ts # Désemballe { success, data } + normalise erreurs │ │ ├──
error.interceptor.ts # Refresh token automatique (401) │ │ └── jwt.interceptor.ts #
Injecte Authorization: Bearer │ ├── guards/ │ │ └── auth.guard.ts # 3 chemins : auth /
lock-screen / signin │ ├── models/ │ │ ├── auth.models.ts # LoginRequest, LoginResponse,
User, 2FA… │ │ └── system.models.ts # Package, Company, Agency, Parameter, FeatureFlag │
├── services/ │ │ ├── auth/ │ │ │ └── auth.service.ts # login, 2FA, logout, refresh, reset
password │ │ └── system/ │ │ ├── company.service.ts # CRUD + suspend/activate/cancel/stats
│ │ ├── agency.service.ts # CRUD + tree │ │ ├── package.service.ts # CRUD + delete │ │ ├──
parameter.service.ts # list + bulkUpdate │ │ └── feature-flag.service.ts # list + toggle │
└── pages/ │ ├── auth/ │ │ ├── login/ # Page de connexion │ │ ├── two-factor/ #
```

```
Vérification TOTP │ │ ├── forgot-password/ # Demande de reset │ │ ├── reset-password/ #
Confirmation reset │ │ └── lock-screen/ # Écran de verrouillage │ └── system/ │ ├──
companies/ # Gestion des institutions │ ├── agencies/ # Gestion des agences │ ├──
packages/ # Offres commerciales │ ├── parameters/ # Paramètres système (bulk edit) │ └──
feature-flags/ # Activation modules par institution ├── app/ │ ├── app.config.ts #
Bootstrap : intercepteurs, router, providers │ ├── core/helpers/ │ │ └── routes.ts #
Constantes de toutes les routes Angular │ ├── core/service/sidebar/ │ │ └──
sidebar.service.ts # Définition des menus (2 blocs de layout) │ ├── features/pages/ │ │
└── pages.routes.ts # Toutes les routes lazy — notre section en bas │ └── layouts/header/
│ └── header.component.ts # Bouton logout connecté à AuthService └── environments/ └──
environment.ts # apiUrl = http://localhost:8000/api
```

3. Conventions et patterns

Pattern de page liste (tableau)

Toutes les pages de liste suivent la même structure :

COMPOSANT TYPESCRIPT

- `packages: T[] = []` — données brutes de l'API
- `filtered: T[] = []` — données filtrées affichées
- `loading = true` — spinner d'attente
- `form: FormGroup` — formulaire création/édition
- `editId: string | null` — null = création, id = édition
- `load()` — appel API + applyFilter()
- `applyFilter()` — filtre local sur search
- `openCreate() / openEdit()` — prépare le modal
- `submit()` — create ou update selon editId

TEMPLATE HTML

- Page header + bouton principal
- 4 stat cards style `bg-{color} border-0` (pattern employee)
- Tableau avec `@for (item of filtered; track item.id)`
- Actions : `d-flex align-items-center border rounded` sur chaque `<a>`
- Icônes : **Tabler CSS** (`ti ti-pencil` , `ti ti-trash` ...) — pas de Feather JS
- Modaux Bootstrap : `data-bs-toggle="modal"`

Icônes : règle importante

> Le template utilise **Feather Icons** (`data-feather="edit"`) qui nécessitent un appel JS `feather.replace()` après chaque rendu. Dans les boucles `@for` d'Angular, cet appel n'est pas déclenché automatiquement → les icônes sont invisibles.

> **Règle :** dans tous nos composants, utiliser exclusivement les **Tabler Icons CSS** (`<i class="ti ti-nom"></i>`) qui fonctionnent sans JavaScript.

Gestion des erreurs API

TYPE D'ERREUR BACKEND	FORMAT REÇU	CE QU'AFFICHE LE COMPOSANT
`ApplicationError (métier)`	`{ code: "ApplicationError", message: "..." }`	Le `message` directement
`Erreur DRF validation (400)`	`{ code: 400, message: "Une erreur...", details: { field: [...] } }`	Les `details` par champ : `field : message`
`401 Unauthorized`	—	`errorInterceptor` tente le refresh automatiquement
`403 Forbidden`	—	Message générique (permission refusée)

4. Intercepteurs HTTP

Déclarés dans `src/app/app.config.ts` dans cet ordre :

```
withInterceptors([jwtInterceptor, errorInterceptor, apiUnwrapInterceptor])
```

INTERCEPTEUR	RÔLE	DÉTAIL
`jwtInterceptor`	Ajoute le token JWT	Injecte `Authorization: Bearer <access>` sur toutes les requêtes vers `API_BASE` , sauf les URLs publiques (login, refresh, reset...)
`errorInterceptor`	Refresh automatique	Sur 401, tente un refresh via `HttpBackend` (bypass intercepteurs pour éviter boucle). Utilise `map(res => res.data)` pour désemballer manuellement la réponse refresh (non interceptée par apiUnwrap). File les requêtes en attente pendant le refresh.
`apiUnwrapInterceptor`	Désemballe + normalise	**Success :** extrait `body.data` de l'enveloppe `{ success, data }` **Erreur :** extrait le message lisible depuis `error.error.message` ou formate les détails de validation DRF champ par champ

Note : errorInterceptor utilise HttpBackend

L' `errorInterceptor` injecte `HttpBackend` (et non `HttpClient`) pour la requête de refresh, afin d'éviter la dépendance circulaire. En conséquence, cette requête **bypasse tous les intercepteurs** y compris `apiUnwrapInterceptor`. Le désemballe de la réponse refresh est donc fait manuellement :

```
http.post<{ success: boolean; data: TokenPair }>(AUTH.refresh, { refresh })
  .pipe(map(res => res.data))
```

5. Module 5 — Authentification Terminé

Pages implémentées

PAGE	ROUTE	FICHIER	STATUT
Login	/signin	_api/pages/auth/login/	✓
Vérification 2FA	/two-step-verification	_api/pages/auth/two-factor/	✓
Mot de passe oublié	/forgot-password	_api/pages/auth/forgot-password/	✓
Réinitialisation	/reset-password	_api/pages/auth/reset-password/	✓
Écran de verrouillage	/lock-screen	_api/pages/auth/lock-screen/	✓

AuthService — méthodes principales

MÉTHODE	ENDPOINT	DESCRIPTION
`login(email, password)`	POST /auth/login/	Retourne tokens ou déclenche 2FA (`requires_2fa: true`). Stocke `user_id` temporaire pour l'étape 2FA.
`loginWith2FA(code)`	POST /auth/login/2fa/	Envoie `{ user_id, code }` . Complète la connexion.
`logout()`	POST /auth/logout/	Blackliste le refresh token. Nettoie TokenService et redirige vers /signin.
`forgotPassword(email)`	POST /auth/password-reset/	Déclenche l'envoi d'email (console backend en dev).
`confirmReset(token, password, confirm)`	POST /auth/password-reset/confirm/	Envoie `{ token, new_password, confirm_password }` .
`unlock(password)`	POST /auth/login/	Revalide le mot de passe depuis l'écran de verrouillage.

TokenService — stockage

CLÉ LOCALSTORAGE	VALEUR
`access_token`	JWT access (durée courte)
`refresh_token`	JWT refresh (durée longue)
`user`	Objet User sérialisé (profil)
`lock_return_url`	URL à restaurer après déverrouillage

authGuard — logique de navigation

```
Accès route protégée
    ├── isAuthenticated() → true       → Accès autorisé
    ├── getUser() → profil en cache → /lock-screen (tokens expirés mais profil connu)
    └── Aucun des deux                 → /signin?returnUrl=...
```

Flux 2FA

1. Login → backend retourne `{ requires_2fa: true, user_id: "uuid" }`
2. `AuthService` stocke `_pending2faUserId` en mémoire et redirige vers `/two-step-verification`

3. L'utilisateur saisit le code TOTP → `loginWith2FA(code)` envoie `{ user_id, code }`
4. Succès → tokens reçus, connexion complète

Points en attente (Auth)

- Envoi réel d'email pour reset password (backend : `services.py` TODO)
- Pages de profil utilisateur (PATCH /auth/me/)
- Gestion des sessions actives (/auth/sessions/)
- Changement de mot de passe depuis les paramètres (/auth/change-password/)
- Interface de configuration 2FA depuis les paramètres

6. Module 6 — Config Système **En cours**

Pages et routes

PAGE	ROUTE ANGULAR	ENDPOINT PRINCIPAL	STATUT
`Packages`	/super-admin/packages	GET/POST /system/packages/admin/	**Tests en cours**
`Companies`	/super-admin/companies	GET/POST /system/companies/	**Tests en cours**
`Agences`	/super-admin/agencies	GET /system/agencies/	**Tests en cours**
`Paramètres`	/settings/system-settings/parameters	GET/POST /system/parameters/bulk/	**À tester**
`Feature Flags`	/settings/system-settings/feature-flags	GET + POST /toggle/	**À tester**

Navigation — Section "Config Système"

Une section dédiée **Config Système** a été ajoutée dans la sidebar (`sidebar.service.ts`), indépendante des menus du template. Elle apparaît dans les deux blocs de layout (vertical et horizontal).

ENTRÉE MENU	ROUTE
`Companies`	/super-admin/companies
`Agences`	/super-admin/agencies
`Packages`	/super-admin/packages
`Paramètres`	/settings/system-settings/parameters
`Feature Flags`	/settings/system-settings/feature-flags

Page Packages — détail d'implémentation

FONCTIONNALITÉS

- 4 stat cards (Total / Actifs / Inactifs / Avec API)
- Tableau trié par date de création desc (`-created_at`)
- Recherche par nom en temps réel
- Création et édition via modal Bootstrap
- Toggle Actif / Accès API dans le formulaire
- Désactivation (soft) via modal de confirmation
- Affichage de tous les packages (actifs + inactifs) pour l'admin

POINTS NOTABLES

- L'unicité est sur `name` (pas `tier`) — plusieurs packages peuvent partager le même tier
- L'admin utilise `GET /packages/admin/` (tous), le public `GET /packages/` (actifs seulement)
- La "suppression" est une désactivation (`is_active = False`) — le backend ne supprime pas physiquement
- Les champs texte optionnels sont assainis (`?? ''`) avant envoi pour éviter `null` sur les `TextField` Django

Modèles TypeScript (system.models.ts)

INTERFACE	CHAMPS PRINCIPAUX
`Package`	id, name, tier, max_users, max_agencies, price_monthly, price_yearly, has_api_access, is_active, description?
`PackageForm`	name, tier, max_users, max_agencies, price_monthly, price_yearly, has_api_access, is_active, description?
`Company`	id, name, slug, email, phone, status, package?, subscription?
`CompanyForm`	name, email, phone, package_id
`Agency`	id, name, company, parent?, type, is_active
`AgencyForm`	name, company, parent?, type
`Parameter`	key, label?, value, data_type, scope, is_active
`FeatureFlag`	id, key, label?, description?, is_enabled

7. Routing et guards

Organisation des routes

FICHIER	RÔLE
`src/app/features/pages/pages.routes.ts`	Routes lazy de toutes les pages (template + nos composants)
`src/app/core/helpers/routes.ts`	Constantes string pour toutes les routes (utilisées dans sidebar et composants)

Constantes de routes ajoutées

CONSTANTE	VALEUR
`routes.superadminCompanies`	/super-admin/companies
`routes.superadminPackages`	/super-admin/packages
`routes.superadminAgencies`	/super-admin/agencies
`routes.systemParameters`	/settings/system-settings/parameters
`routes.systemFeatureFlags`	/settings/system-settings/feature-flags

Guards

GUARD	FICHIER	COMPORTEMENT
`authGuard`	_api/guards/auth.guard.ts	Protège les routes authentifiées. Redirige vers /lock-screen si profil en cache mais tokens expirés, sinon vers /signin.
`guestGuard`	Template (existant)	Redirige les utilisateurs connectés qui tentent d'accéder à /signin vers le dashboard.

8. Sidebar et navigation

RÈGLE D'OR

Les menus du template (`sidebar.service.ts`) sont laissés intacts. Nos entrées sont ajoutées dans une **section dédiée "Config Système"** dans les deux blocs de définition de la sidebar (layout vertical ~ligne 120, layout horizontal ~ligne 2166).

La sidebar contient deux blocs distincts (deux layouts) qui doivent être mis à jour en parallèle :

- **Bloc 1** (~ligne 85) : layout vertical, propriétés `icon2`
- **Bloc 2** (~ligne 2126) : layout horizontal/icônes, propriétés `icon + materialicons + ids`

9. Modules à venir

Module 5 — Authentification ✓

Login, 2FA, logout, lock screen, forgot/reset password, authGuard

Module 6 — Config Système (en cours)

Packages, Companies, Agences, Paramètres, Feature Flags — tests en cours

Module 7 — Admin RBAC

Rôles, permissions, groupes, assignation utilisateurs

Module 8 — Gestion Utilisateurs

CRUD utilisateurs, profils, changement de mot de passe, sessions

Module 9 — RH (Ressources Humaines)

Employés, départements, postes, contrats

Module 10 — Flotte

Véhicules, types de véhicules, maintenance

Module 11 — Config Transport

Lignes, gares, arrêts, tarifs

Module 12 — Voyages

Départs, conducteurs, affectations véhicules

Module 13 — Ticketing

Réservations, billets, bagages, remboursements

Module 14 — Colis

Envois, suivi, tarification colis

Module 15 — Trésorerie

Caisses, mouvements, rapprochements

Module 16 — Réservation en ligne

API publique, paiement MoMo, confirmation

Ce document est mis à jour progressivement au fil du développement.
Dernière mise à jour : Juin 2026 — Module 6 en cours de test.